DATA STRUCTURES AND ALGORITHMS

SEARCHX

C++ Mini Search Engine

PROJECT REPORT

University	
Department	
Course Name	Data Structures and Algorithms
Course Code	
Project Title	SearchX — C++ Mini Search Engine
Student Name	
Student ID / Roll	
Semester / Section	
Submitted To	
Submission Date	

Table of Contents

1	Abstract
2	Introduction
3	Project Objectives
4	System Architecture
5	Project Workflow
6	File and Module Structure
7	Class Structure
8	Data Structures Used
9	Algorithms Used
10	Indexing and Text Processing
11	Search Operation
12	Search Algorithms Module
13	Sorting Algorithms Module
14	Document Graph
15	Search History and Recent Searches
16	Sparse Matrix
17	Index Lookup (“Hash Search”) Note
18	Time and Space Complexity
19	Object-Oriented Programming Concepts
20	User Interface and Menu System
21	Sample Input and Output
22	Testing
23	Error Handling
24	Advantages
25	Limitations
26	Future Improvements
27	Coverage of DSA Topics
28	Conclusion
29	References
30	Appendix

1. Abstract

SearchX is a console-based mini search engine written in C++ that searches four local text documents (`dsa.txt`, `cpp.txt`, `algorithms.txt` and `database.txt`). It was chosen as a Data Structures and Algorithms project because a search engine needs many different data structures to work together: text must be cleaned, words must be stored and looked up quickly, results must be ranked, and relationships between documents must be represented.

When the program starts, a `TextProcessor` reads each document, converts every word to lowercase and removes non-alphanumeric characters. Each cleaned word is stored in an **inverted index** (an array of 27 linked-list buckets in which every word keeps a linked list of the documents that contain it together with its frequency) and in a **Binary Search Tree** that keeps the unique vocabulary in alphabetical order (84 unique words in the sample data). A search query is cleaned in the same way, and each document receives a score equal to the sum of the frequencies of the query words; the scores are ranked using a **Max Heap**.

Further modules demonstrate other DSA topics: Linear Search, Binary Search and index-based lookup with comparison counts; Bubble, Selection and Insertion Sort with comparison counts; a **weighted undirected graph** of documents (edge weight = number of shared distinct words) traversed with BFS, DFS and a BFS shortest path; a **sparse matrix** of document–term frequencies; a **stack** for search history; and a **deque** for recent searches. SearchX therefore shows, in one integrated application, how data structures and algorithms solve a practical problem.

2. Introduction

2.1 Background

A search engine takes a user query, finds which documents contain the query terms and returns them in order of relevance. Instead of scanning every document for every query, search engines build an **index** in advance — usually an *inverted index*, which maps each word to the documents containing it (like the index at the back of a book).

2.2 Problem Statement

Searching for a word manually across many documents means opening each file and scanning it. This is slow, error-prone, gives no ranking and shows no relation between documents.

2.3 Proposed Solution

SearchX reads all documents once, builds an inverted index and a sorted vocabulary, and answers queries from memory. Documents are ranked with a max heap, and a document graph and a sparse matrix present the relations between documents and words.

2.4 Motivation

The project connects classroom topics (linked lists, trees, heaps, graphs, sorting, searching and complexity analysis) to one realistic application instead of isolated exercises.

2.5 Scope

Can do (implemented)	Cannot do (limitations)
Index 4 fixed `.txt` files from the `documents/` folder	Load arbitrary or new documents at run time (file list is fixed)
Single- and multi-word queries with frequency-based ranking	Phrase search, AND/OR operators, stop-word removal, TF-IDF
Console menu interface	GUI or web interface
All data held in memory	Saving the index or history to disk

3. Project Objectives

#	Objective	Realised by
1	Apply DSA to a real-world-style problem	Whole SearchX system
2	Implement document indexing	`InvertedIndex`, `SearchIndex`
3	Perform searching	`SearchEngine::search`, `SearchAlgorithms`
4	Maintain a sorted vocabulary	`BST`
5	Implement sorting	`SortingAlgorithms`
6	Represent document relationships	`Graph`
7	Store sparse information efficiently	`SparseMatrix`
8	Maintain search history	`SearchHistory` (stack), `RecentSearchDeque`
9	Rank results	`MaxHeap`
10	Integrate many structures in one OOP application	`SearchEngine` composition

4. System Architecture

SearchX is a single-process console application. `main()` owns one `SearchEngine` object, which acts as the coordinator. The components do not call each other sideways: `SearchEngine` calls `SearchIndex`, `Graph`, `MaxHeap` and the other modules, and results are printed through `cout` and the `UI` helpers.

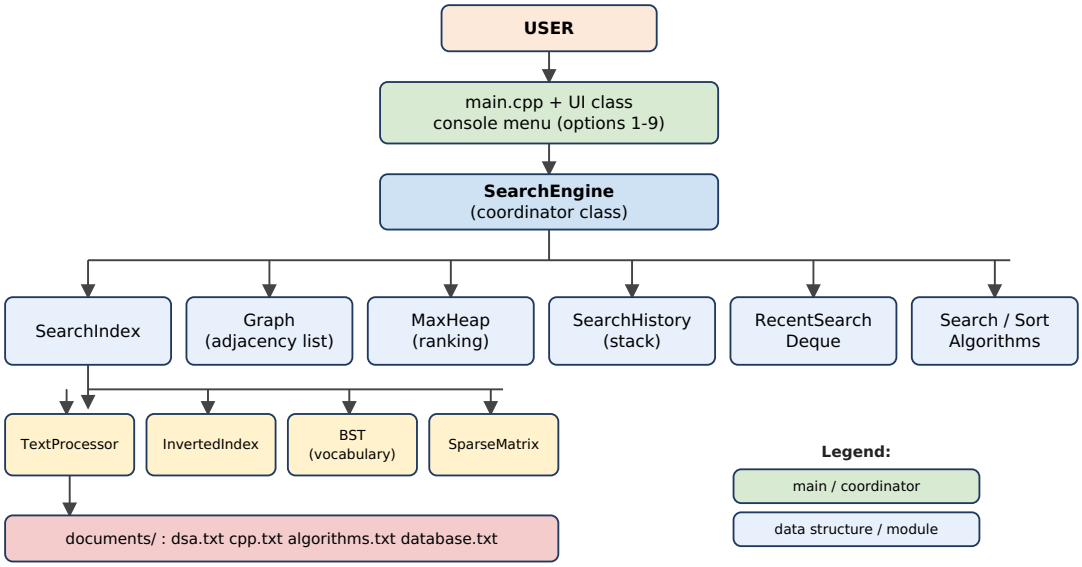


Figure 1: Overall architecture of SearchX

Member of SearchEngine	Type	Purpose
`fileNames[4]`, `documentCount`	string array, int	Fixed document list (count = 4)
`searchIndex`	`SearchIndex`	Index, vocabulary and sparse matrix
`documentGraph`	`Graph*` (dynamic)	Document relationship graph
`history`	`SearchHistory`	Stack of queries
`recentSearches`	`RecentSearchDeque`	Deque of queries
`algorithms`	`SearchAlgorithms`	Linear / binary search
`sorter`	`SortingAlgorithms`	Sorting algorithms

`SearchIndex` in turn contains a `TextProcessor`, an `InvertedIndex` and a `BST` (vocabulary).

5. Project Workflow

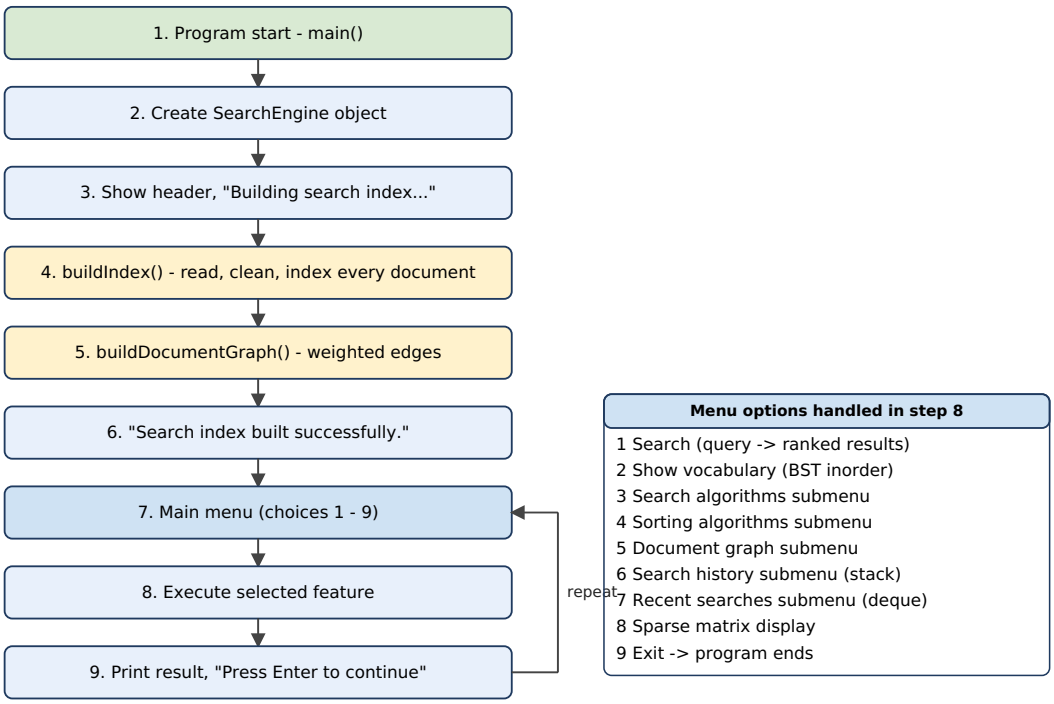


Figure 2: Complete program workflow

Stage explanation

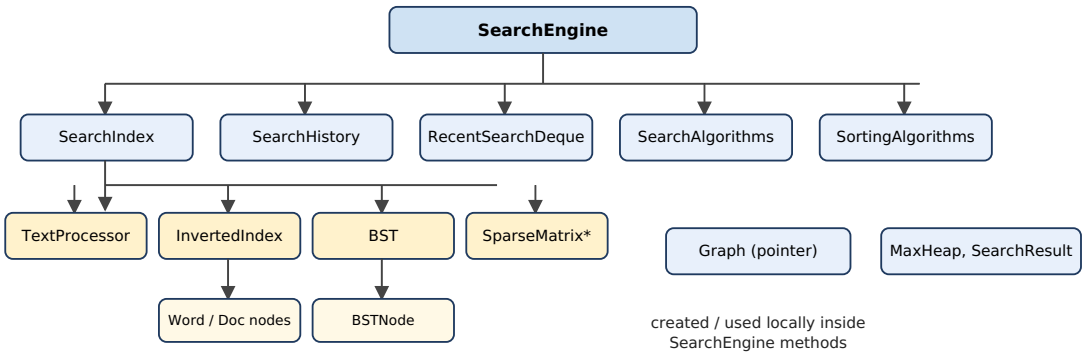
1. **Start / construct:** the `SearchEngine` object is created; the four file names are set, `documentCount` = 4 and the graph pointer is `NULL` until the index is built.
2. **Load documents:** for each file the path `documents/<fileName>` is built and `TextProcessor::processDocument` returns the list of cleaned words.
3. **Process text:** each whitespace-separated token is lowercased and stripped of non-alphanumeric characters; empty results are discarded.
4. **Build index:** every word occurrence is inserted into the `InvertedIndex` (which increments the frequency if the word–document pair already exists) and into the BST (duplicates ignored).
5. **Build graph:** every pair of documents is compared and an edge is added when they share at least one distinct word; the edge weight is the number of shared distinct words.
6. **Menu loop:** the choice is read with `getline` and the matching function is called until option 9 is chosen.

6. File and Module Structure

File	Purpose	Important functions
`main.cpp`	Program entry and main menu loop	`main()`
`SearchEngine.cpp`	Coordinator; menus and search logic	`buildIndex`, `search`, `showVocabulary`, `searchAlgorithmMenu`, `sortingAlgorithmMenu`, `buildDocumentGraph`, `graphMenu`, `historyMenu`, `recentSearchMenu`, `sparseMatrixMenu`
`SearchIndex.cpp`	Combines text processing, inverted index, vocabulary and sparse matrix	`addDocument`, `buildIndex`, `contains`, `getFrequency`, `getVocabulary`, `displayVocabulary`, `buildSparseMatrix`, `displaySparseMatrix`
`TextProcessor.cpp`	Text cleaning and tokenising	`cleanWord`, `processDocument`, `processQuery`
`InvertedIndex.cpp`	Word → documents → frequency	`getIndex`, `findWord`, `addDocument`, `insert`, `search`, `displayWord`, `getFrequency`
`BST.cpp`	Sorted unique vocabulary	`insert`, `search`, `displayInorder`, `getVocabulary`, `size`
`Heap.cpp`	Max heap for ranking	`heapifyUp`, `heapifyDown`, `insert`, `extractMax`
`SearchResult.cpp`	Result record (file name, score)	`SearchResult(fileName, score)`
`Graph.cpp`	Weighted undirected document graph	`addEdge`, `displayGraph`, `BFS`, `DFS`, `shortestPath`
`SparseMatrix.cpp`	Linked-list sparse matrix	`insert`, `getValue`, `displayNamed`, `nonZeroCount`

`SearchHistory.cpp`	Stack of past queries	`push`, `pop`, `peek`, `size`, `display`
`RecentSearchDeque.cpp`	Doubly linked deque of recent queries	`pushFront`, `pushBack`, `popFront`, `popBack`, `display`
`SearchAlgorithms.cpp`	Linear / binary search and sort helper	`linearSearch`, `binarySearch`, `sortData`
`SortingAlgorithms.cpp`	Sorting with comparison counting	`bubbleSort`, `selectionSort`, `insertionSort`
`LinkedList.cpp`	Reusable basic singly linked list of strings	`insert`, `search`, `display`, `size`
`UI.cpp`	Console helpers	`clearScreen`, `pause`, `showHeader`, `showMessage`

7. Class Structure



Arrows = "has-a" (composition) relationship. * SparseMatrix is created temporarily when menu 8 is opened.

Figure 3: Class relationship (composition) diagram

Class	Purpose and key members
SearchEngine	Central coordinator and menu logic. Owns the index, graph (via pointer, deleted in destructor), history stack, recent-search deque, search and sort helpers. `search(query)` cleans the query, records history, scores documents, ranks with `MaxHeap` and prints.
SearchIndex	One place that builds and queries everything related to the index. Members: `TextProcessor`, `InvertedIndex`, `BST`. The sparse matrix is created as a local object each time menu 8 is opened.
TextProcessor	Stateless utility. `cleanWord` lowercases and keeps only alphanumeric characters; `processDocument` reads a file and tokenises it; `processQuery` does the same for a query string.
InvertedIndex	Array `wordLists[27]` of linked lists of word nodes; each word node holds a linked list of document nodes (file name, frequency). New nodes are inserted at the head.
BST	Recursive binary search tree of strings. Duplicates ignored. Provides inorder display and inorder-to-vector. Not self-balancing.
Graph	Adjacency-list graph: `vertices`, `vertexNames[]`, array of edge lists (neighbour, weight). `addEdge` inserts both directions.
SparseMatrix	Linked list of non-zero nodes (row, column, value) sorted in row-major order.
MaxHeap / SearchResult	Vector-based max heap of `SearchResult` objects (file name, score).
SearchHistory	Linked-list stack (LIFO): `top`, `count`.
RecentSearchDeque	Doubly linked list with `front`, `rear`, `count`; nodes hold query, next, previous.
SearchAlgorithms / SortingAlgorithms	Stateless method collections; each algorithm reports a comparison count through a reference parameter.
UI	Console helpers (`system("cls")` on Windows, `system("clear")` otherwise).

8. Data Structures Used

Notation: D = number of documents (4), V = unique words (84 in the sample data), N = total word occurrences, B = length of one bucket chain, d = documents containing a word, h = BST height, Q = words in the query, k = matching documents.

#	Data structure	File	Role in SearchX
1	Array of linked lists (27 buckets)	`InvertedIndex.cpp`	Maps word → (document, frequency)
2	Binary Search Tree	`BST.cpp`	Sorted unique vocabulary
3	Max Heap (vector based)	`Heap.cpp`	Ranks matching documents
4	Graph (adjacency list, undirected, weighted)	`Graph.cpp`	Documents = vertices, shared words = edge weight
5	Sparse matrix (sorted linked list)	`SparseMatrix.cpp`	Document × term frequency table
6	Stack (linked list)	`SearchHistory.cpp`	Search history
7	Deque (doubly linked list)	`RecentSearchDeque.cpp`	Recent searches
8	Singly linked list class	`LinkedList.cpp`	Reusable basic structure (linked-list concept is also used inside the index, sparse matrix, stack, deque and graph)
9	Dynamic arrays and `std::vector`	several files	BFS queue, visited arrays, word lists

8.1 Inverted Index (array of linked lists)

Definition: a structure that maps each term to the documents that contain it. **Why needed:** it avoids rescanning all documents on every query.

Implementation: an array of 27 slots; slots 0–25 are for words starting with a–z and slot 26 for digits/other/empty words. Each slot is a linked list of word nodes; each word has a linked list of document nodes with a frequency counter.

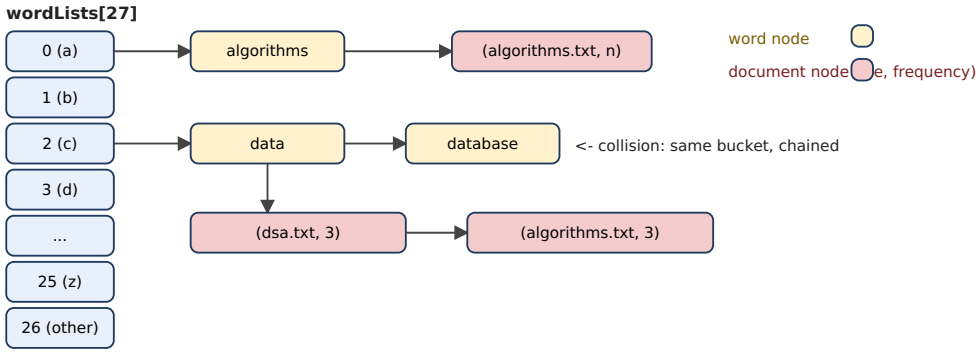


Figure 4: Inverted index structure (bucket by first letter, chained word nodes, document list per word)

Operation	Working
`insert`	Find bucket → scan chain for the word → if found, `addDocument` (increment frequency or add a document node at the head) → if not found, create word node, add document and put it at the head of the bucket
`search` / `findWord`	Bucket by first letter → scan chain comparing strings
`getFrequency`	`findWord` → scan its document list
`displayWord`	Prints the word and each document with its count

Time: insert O(B + d); search O(B); getFrequency O(B + d); in the worst case (all words share one first letter) B approaches V. **Space:** O(number of distinct word–document pairs + V), i.e. O(N). **Example from the program output:** the word `data` occurs 3 times in `dsa.txt` and 3 times in `algorithms.txt`.

8.2 Binary Search Tree (vocabulary)

A binary tree in which left < node < right. Every token from every document is inserted (duplicates ignored). An inorder traversal gives the vocabulary in alphabetical order; `size()` gives "Total unique words: 84".

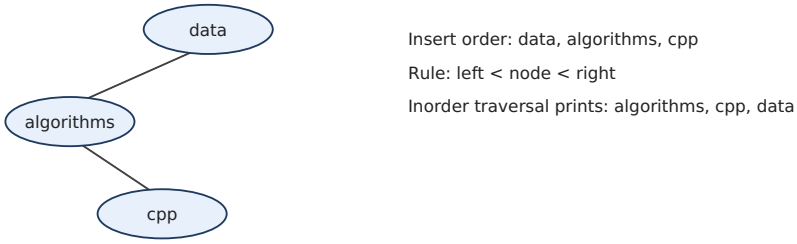


Figure 5: BST example (insertion of data, algorithms, cpp)

Operations: insert, search, inorder display, inorder-to-vector, count and destroy (post-order). **Time:** insert/search O(h) — average O(log V), worst case O(V) when words arrive almost sorted (the tree is not self-balancing); inorder traversal O(V). **Space:** O(V) nodes plus O(h) recursion.

8.3 Graph

Vertices are documents; an edge exists when two documents share at least one distinct word; the edge weight is the number of such shared distinct words. `addEdge` creates two edge nodes (undirected). Operations: display, BFS, DFS and shortest path.

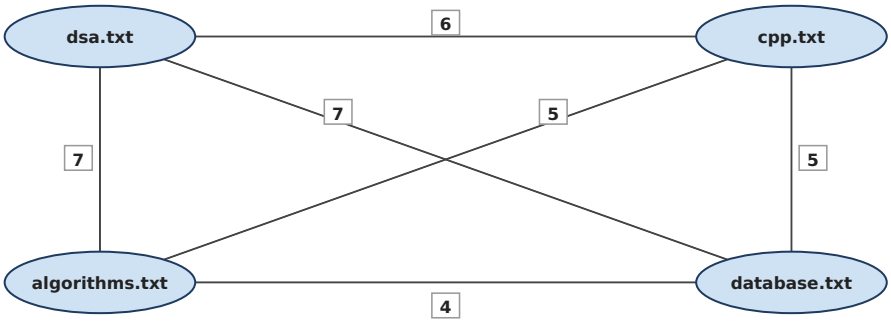


Figure 6: Document graph built from the four sample documents (edge weight = shared distinct words)

Pair	Weight
dsa – cpp	6
dsa – algorithms	7
dsa – database	7
cpp – algorithms	5
cpp – database	5
algorithms – database	4

Every pair of documents shares words, so the sample graph is a complete graph on 4 vertices. **Time:** addEdge O(1); BFS/DFS and shortest path $O(V_g + E)$ where $V_g = 4$. **Space:** $O(V_g + E)$.

8.4 Sparse Matrix

Rows are documents, columns are vocabulary terms (position in the sorted vocabulary) and the value is the frequency. Only non-zero entries are stored as nodes of a list sorted by (row, column). A dense table would need 4 × 84 = 336 cells, whereas the sparse list stores only the (document, word) pairs that actually occur.

```
Dense (3 docs x 5 terms)        Sparse list (row, col, value)
[ 2 0 0 1 0 ]                   (0,0,2) -> (0,3,1) -> (1,1,3) -> (2,4,1)
[ 0 3 0 0 0 ]
[ 0 0 0 0 1 ]
```

Illustration of dense versus sparse storage (small example numbers)

8.5 Max Heap

A complete binary tree stored in a `vector<SearchResult>` where every parent ≥ its children (parent of i = (i−1)/2; children 2i+1 and 2i+2). In `SearchEngine::search` every document with score > 0 is inserted; results are printed by calling `extractMax()` until the heap is empty. Insert = push_back + heapifyUp; extractMax = take root, move last element to root, heapifyDown.

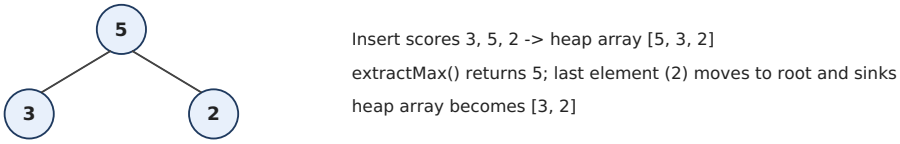


Figure 7: Max-heap insert and extractMax example

Time: insert and extractMax O(log k), printing all k results is O(k log k) (effectively heap sort). **Space:** O(k). Documents with equal scores have no guaranteed order because a heap is not stable.

8.6 Stack and Deque

- **Stack** (`SearchHistory`): LIFO linked list; push, pop and peek are O(1); display is O(n).
- **Deque** (`RecentSearchDeque`): doubly linked list; push and pop at both ends are O(1); display is O(n).

8.7 Linked List

`LinkedList` is implemented as a reusable basic structure (head pointer; insert at tail O(n); search O(n); display; size). The linked-list concept itself is used inside `InvertedIndex`, `SparseMatrix`, `SearchHistory`, `RecentSearchDeque` and the graph adjacency lists, each with its own node type.

9. Algorithms Used

Algorithm	File	Used by
Linear Search	`SearchAlgorithms.cpp`	Menu 3 → 1
Binary Search	`SearchAlgorithms.cpp`	Menu 3 → 2
Index (bucket) lookup	`InvertedIndex.cpp` via `SearchIndex::contains`	Menu 3 → 3
Bubble Sort (counts comparisons)	`SortingAlgorithms.cpp`	Menu 4 → 1
Selection Sort	`SortingAlgorithms.cpp`	Menu 4 → 2
Insertion Sort	`SortingAlgorithms.cpp`	Menu 4 → 3
Bubble-sort helper `sortData`	`SearchAlgorithms.cpp`	Prepares data for binary search
BST insert / search / inorder traversal	`BST.cpp`	Vocabulary
Heap: heapifyUp, heapifyDown, insert, extractMax	`Heap.cpp`	Search ranking
BFS, recursive DFS, BFS shortest path (fewest edges)	`Graph.cpp`	Menu 5
Document similarity (distinct common words)	`SearchEngine::buildDocumentGraph`	Graph edge weights
Term-frequency scoring (sum of frequencies)	`SearchEngine::search`	Ranking

9.1 Linear Search

```
comparisons = 0
for i = 0 to n-1:
    comparisons++
    if data[i] == target: return true
return false
```

Best O(1), average/worst O(n). In the program output the word `data` was found with **18 comparisons** (it is the 18th word of the sorted vocabulary).

9.2 Binary Search

Precondition: sorted data. The menu sorts a copy of the vocabulary first with `sortData` (bubble sort).

```
left = 0, right = n-1
while left <= right:
    mid = (left + right) / 2
    comparisons++
    if data[mid] == target: return true
    if target < data[mid]: right = mid-1 else left = mid+1
return false
```

Best O(1), average/worst O(log n); for 84 words at most $\lfloor \log_2 84 \rfloor + 1 = 7$ comparisons. The program output shows **5 comparisons** for `data`. The count excludes the sorting step, which is $O(n^2)$.

9.3 Bubble Sort

```
for i = 0 to n-2:
    for j = 0 to n-i-2:
        comparisons++
        if data[j] > data[j+1]: swap
```

There is no early exit, so it always performs $n(n-1)/2$ comparisons: $84 \times 83 / 2 =$ **3486**, matching the program output.

9.4 Selection Sort

```
for i = 0 to n-2:
    min = i
    for j = i+1 to n-1:
        comparisons++
        if data[j] < data[min]: min = j
    if min != i: swap(data[i], data[min])
```

Always 3486 comparisons for n = 84, matching the program output.

9.5 Insertion Sort

```
for i = 1 to n-1:
    key = data[i]; j = i-1
    while j >= 0:
        comparisons++
        if data[j] <= key: break
        data[j+1] = data[j]; j--
    data[j+1] = key
```

The vocabulary arrives already sorted (BST inorder), which is the **best case**: $n-1$ = **83** comparisons, matching the program output.

9.6 Breadth-First Search (BFS)

```
visited[all] = false; queue = [start]; visited[start] = true
while queue not empty:
    u = dequeue; print u
    for each neighbour v of u:
        if not visited[v]: visited[v] = true; enqueue v
```

Array-based queue of size V_g; time $O(V_g + E)$.

9.7 Depth-First Search (DFS, recursive)

```
DFS(u): visited[u] = true; print u
        for each neighbour v of u:
            if not visited[v]: DFS(v)
```

Time $O(V_g + E)$; recursion depth at most V_g.

9.8 Shortest Path (BFS)

BFS is run from the start vertex while recording `parent[]`; the search stops when the destination is dequeued and the path is rebuilt by following parents back from the destination. Edge weights are ignored, so this finds the path with the **fewest edges**, not the minimum total weight (it is not Dijkstra's algorithm). If no path exists, "No path exists." is printed.

9.9 Document similarity and ranking score

- **Similarity (graph construction):** for each pair (i, j), count the distinct words of document i that also exist in document j. Time roughly $O(D^2 \cdot L^2)$ for L words per document — fine for four small files.
- **Ranking score:** score(doc) = Σ frequency(query word, doc) over all query words (raw term-frequency summation).

10. Indexing and Text Processing

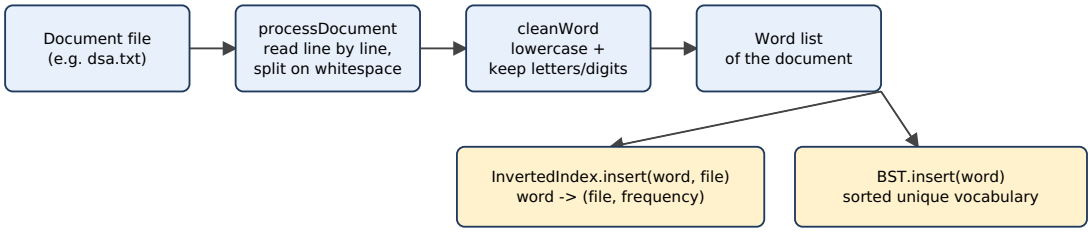


Figure 8: Document indexing pipeline

cleanWord processes each character: convert to lowercase with `tolower` and keep it only if `isalnum` is true. Tokenising is done on whitespace only (`stringstream >> word`).

Original token	Result	Reason
`Algorithms,`	`algorithms`	lowercase, comma removed
`Structures!`	`structures`	`!` removed
`C++`	`c`	`+` removed
`data-structures`	`datastructures`	hyphen removed inside the token (no splitting)
`...`	(empty, discarded)	no alphanumeric characters
`C99`	`c99`	digits are kept

Consequences: there is no stemming (`algorithm` and `algorithms` are separate vocabulary words), no stop-word removal (`a`, `an`, `and` are indexed) and hyphenated words are merged.

Illustrative indexing example (small invented text)

Document	Text	Cleaned words
D1 `cpp.txt`	“C++ algorithms”	c, algorithms
D2 `dsa.txt`	“Data structures, data!”	data, structures, data
D3 `database.txt`	“Data base”	data, base

Word	Documents (frequency)
algorithms	D1 (1)
base	D3 (1)
c	D1 (1)
data	D2 (2), D3 (1)
structures	D2 (1)

Query `data` → bucket ‘d’ → word `data` → D2 score 2, D3 score 1 → the heap ranks D2 first.

11. Search Operation

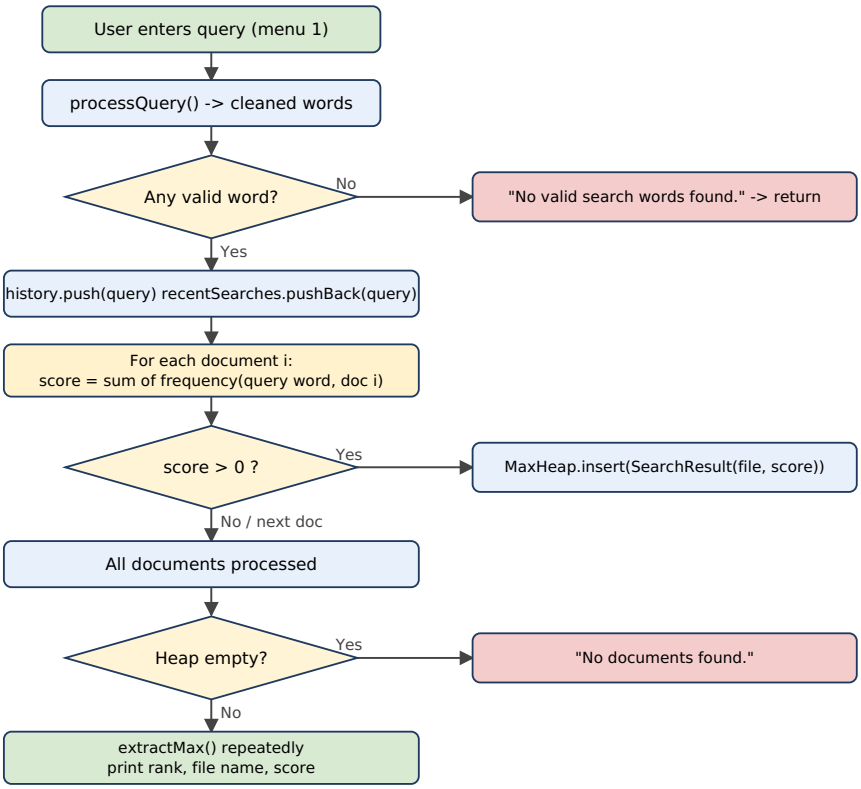


Figure 9: Flowchart of SearchEngine::search()

- Multi-word queries behave like **OR** with summed scores.
- The original query string (not the cleaned one) is stored in the history; an invalid or empty query is not stored.
- **Complexity:** O(D · Q · (B + d)) for scoring plus O(k log k) for ranking.

12. Search Algorithms Module

Menu: `1. Linear Search 2. Binary Search 3. Hash Search 4. Back`. The entered word is cleaned with `cleanWord`; an empty result prints “Invalid word.”

Aspect	Linear	Binary	Index lookup
Data searched	vocabulary vector	sorted copy of vocabulary	inverted-index buckets
Needs sorted data	No	Yes	No
Best case	O(1)	O(1)	O(1)
Average case	O(n)	O(log n)	O(B), short chains
Worst case	O(n)	O(log n)	O(V) if all words share one letter
Extra cost	copy O(n)	copy + bubble sort $O(n^2)$	none
Result for `data`	18 comparisons	5 comparisons	Found

13. Sorting Algorithms Module

Menu: `1. Bubble Sort 2. Selection Sort 3. Insertion Sort 4. Back`. A copy of the vocabulary is sorted; the program prints the algorithm name, the comparison count and the sorted list.

Algorithm	Best	Average	Worst	Space	Stable	Comparisons (n = 84, sorted input)
Bubble (no early exit)	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	Yes	3486
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No (swap version)	3486
Insertion	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	83

Observation: the data is already sorted (BST inorder), so insertion sort shows its best case while bubble and selection sort do not benefit from the ordering.

14. Document Graph

- **Vertex:** a document (index 0–3 → `dsa.txt`, `cpp.txt`, `algorithms.txt`, `database.txt`).
- **Edge / weight:** two documents are connected if they share at least one distinct word; weight = number of shared distinct words.
- **Construction:** `buildDocumentGraph()` names the vertices, then compares each pair (i < j) and calls `addEdge(i, j, commonWords)`.
- **Why useful:** shows which documents are most related and allows BFS, DFS and shortest-path demonstrations.

Adjacency list produced by the program (new edges are inserted at the head of each list):

```
dsa.txt        -> database.txt(7) -> algorithms.txt(7) -> cpp.txt(6)
cpp.txt        -> database.txt(5) -> algorithms.txt(5) -> dsa.txt(6)
algorithms.txt -> database.txt(4) -> cpp.txt(5)        -> dsa.txt(7)
database.txt   -> algorithms.txt(4)-> cpp.txt(5)       -> dsa.txt(7)
```

- **BFS from dsa.txt:** dsa.txt, database.txt, algorithms.txt, cpp.txt.
- **DFS from dsa.txt:** dsa.txt → database.txt → algorithms.txt → cpp.txt (same order here because of the data).
- **Shortest path dsa.txt → database.txt:** direct edge, 1 edge. In this sample every pair is directly connected, so every shortest path has 1 edge.

Menu input is validated with `try/catch` around `stoi` and range checks (“Invalid document choice.”).

15. Search History and Recent Searches

15.1 Search History (stack)

- **Structure:** singly linked list used as a stack (`top`, `count`). `push` makes the new node the top (O(1)) and is called from `search()` for every valid query.
- **Access:** `peek` returns the latest query; `pop` removes it; `display` walks from the top (most recent first).
- **Menu:** 1 Show History, 2 Remove Latest Search, 3 Show Latest Search, 4 Show History Size, 5 Back.
- **Program output:** history `data, code, dsa, hello` (latest first); removing the latest printed “Removed search: data”; afterwards latest = `code` and size = 3.

15.2 Recent Searches (deque)

- **Structure:** doubly linked list with `front` and `rear`. `pushBack` is used for every search, so the front is the oldest and the rear is the newest query.
- **Operations:** pushFront, pushBack, popFront, popBack, getFront, getBack, empty, size, display — all O(1) except display O(n).
- **Why a deque:** both ends are accessible, so the menu can remove the oldest (`popFront`) or the latest (`popBack`) search in constant time.
- **Example:** searches `dsa`, `code` → `[dsa, code]`; popFront returns `dsa`; popBack returns `code`; count = 0.
- **Difference from history:** history is LIFO (newest first); the recent list is shown oldest first and can be trimmed from either end.

16. Sparse Matrix Module

`SearchIndex::buildSparseMatrix` obtains the vocabulary, creates `SparseMatrix(documentCount, V)` and, for each document i and word j, inserts `(i, j, frequency)` when `getFrequency` is greater than 0. `displayNamed` prints each non-zero entry as *Document* | *Word* | *Frequency*, followed by the number of non-zero elements, documents and vocabulary terms.

Sample entries from the program output for `dsa.txt`:

Document	Word	Frequency
dsa.txt	a	2
dsa.txt	and	1
dsa.txt	data	3
dsa.txt	first	3
dsa.txt	follows	2
dsa.txt	in	2

Complexity: insert O(k) and getValue O(k), where k is the number of stored non-zero nodes; because the structure is a plain sorted list (not a CSR format), random access is O(k).

17. Index Lookup (“Hash Search”) Note

The third entry of the Search Algorithms menu is labelled “Hash Search”. The implementation does not use a modulo or polynomial hash function; it uses a **first-character bucket function** with separate chaining (a linked list per bucket):

```
getIndex(word):
    if word is empty -> 26
    c = tolower(word[0])
    if 'a' <= c <= 'z' -> c - 'a'      (0 ... 25)
    else               -> 26           (digits / other)
```

Property	SearchX index	Typical hash table
Key → slot	first letter	hash(key) mod size
Distribution	uneven (many words start with the same letters)	designed to be uniform
Collision handling	chaining (linked list)	chaining or open addressing
Lookup time	O(B)	O(1) average
Resize / rehash	none	usually yes

Example: `data` → ‘d’ − ‘a’ = 3 → bucket 3; `database` also goes to bucket 3 (collision handled by the chain); `algorithms` goes to bucket 0. Choosing the bucket is O(1), but scanning the chain makes the lookup O(B). Replacing this with a true hash function is listed under future improvements.

18. Time and Space Complexity

Operation	Structure / algorithm	Best	Average	Worst	Space
Insert word	InvertedIndex	O(1)	O(B + d)	O(V + d)	O(1) per new node
Lookup word	InvertedIndex	O(1)	O(B)	O(V)	O(1)
Get frequency	InvertedIndex	O(1)	O(B + d)	O(V + d)	O(1)
Insert / search word	BST	O(1)	O(log V)	O(V)	O(h)
Inorder traversal	BST	O(V)	O(V)	O(V)	O(h) + output
Heap insert	MaxHeap	O(1)	O(log k)	O(log k)	O(1)
Heap extractMax	MaxHeap	O(log k)	O(log k)	O(log k)	O(1)
Query scoring	search()	—	O(D·Q·(B+d))	same	O(k)
Linear search	array	O(1)	O(n)	O(n)	O(1) + copy
Binary search	sorted array	O(1)	O(log n)	O(log n)	O(1) + copy
Bubble sort	array	O(n²)	O(n²)	O(n²)	O(1)
Selection sort	array	O(n²)	O(n²)	O(n²)	O(1)
Insertion sort	array	O(n)	O(n²)	O(n²)	O(1)
Add graph edge	adjacency list	O(1)	O(1)	O(1)	O(1)
Build graph	nested loops	—	O(D²·L²)	same	O(D·L + E)
BFS / DFS	adjacency list	O(V_g+E)	O(V_g+E)	O(V_g+E)	O(V_g)
Shortest path	BFS	O(1)	O(V_g+E)	O(V_g+E)	O(V_g)
Sparse insert	linked list	O(1)	O(k)	O(k)	O(1)
Stack push / pop / peek	linked stack	O(1)	O(1)	O(1)	O(1)
Deque push / pop (both ends)	doubly linked	O(1)	O(1)	O(1)	O(1)

Reading the table: because the data set is tiny (4 documents, 84 words) everything runs instantly, but the analysis shows what would happen with larger data: the bucket index and the unbalanced BST can degrade to linear time, bubble sort is quadratic, and graph construction and sparse-matrix building become expensive.

19. Object-Oriented Programming Concepts

Concept	Evidence in SearchX
Classes and objects	`SearchEngine searchEngine;` in `main`; every module is a class
Encapsulation	Data (heads, counts, vectors) is accessed through member functions such as `history.push`
Abstraction	`SearchEngine::search` hides indexing and heap details from `main`
Constructors	`SearchEngine()`, `Graph(int)`, `SparseMatrix(rows, cols)`, `BST()`
Destructors	All dynamic structures free their nodes (`~InvertedIndex`, `~BST`, `~Graph`, `~SparseMatrix`, `~SearchHistory`, `~RecentSearchDeque`, `~LinkedList`, `~SearchEngine`)
Composition (has-a)	`SearchEngine` has `SearchIndex`, history, deque; `SearchIndex` has `BST`, `InvertedIndex`, `TextProcessor`
Dynamic memory	`new` / `delete` for nodes, the `Graph` object and arrays in BFS/DFS
Inheritance / polymorphism	Not used in this project

20. User Interface and Menu System

Console menus using `getline` for input and `UI::pause()` (“Press Enter to continue...”).

Main menu choice	Action
1	Search — asks “Enter search query:”, calls `search`
2	Show Vocabulary — total unique words and sorted list (BST inorder)
3	Search Algorithms submenu
4	Sorting Algorithms submenu
5	Document Graph submenu
6	Search History submenu
7	Recent Searches submenu
8	Sparse Matrix — builds and prints the document-term matrix
9	Exit — prints “Thank you for using SearchX!”
other	“Invalid choice.”

other	Invalid choice.

Submenu	Options
Search Algorithms	1 Linear Search, 2 Binary Search, 3 Hash Search, 4 Back
Sorting Algorithms	1 Bubble Sort, 2 Selection Sort, 3 Insertion Sort, 4 Back
Document Graph	1 Display Graph, 2 BFS, 3 DFS, 4 Shortest Path, 5 Back
Search History	1 Show History, 2 Remove Latest, 3 Show Latest, 4 Show Size, 5 Back
Recent Searches	1 Show Recent, 2 Remove Oldest, 3 Remove Latest, 4 Show Count, 5 Back

21. Sample Input and Output

Example 1 — Starting the program

```
==========================================
                SEARCHX
         C++ MINI SEARCH ENGINE
==========================================

Building search index...
Search index built successfully.

Press Enter to continue...
```

Example 2 — Searching a word

```
Enter search query: data
Query: data
Cleaned words: data
1. dsa.txt
   Score: 3
2. algorithms.txt
   Score: 3
```

An unknown word such as `hello` prints “No documents found.”

Example 3 — Vocabulary

```
Total unique words: 84
Words in alphabetical order:
a, algorithm, algorithms, an, and, are, arrange, arrays, between, binary, by, c, can, classes ...
```

Example 4 — Search algorithms (word: data)

```
Algorithm: Linear Search   Result: Found   Comparisons: 18
Algorithm: Binary Search   Result: Found   Comparisons: 5
```

Example 5 — Sorting algorithms

```
Bubble Sort     Comparisons: 3486
Selection Sort  Comparisons: 3486
Insertion Sort  Comparisons: 83
```

Example 6 — Document graph

```
BFS: dsa.txt database.txt algorithms.txt cpp.txt
DFS: dsa.txt database.txt algorithms.txt cpp.txt
Shortest Path: dsa.txt -> database.txt     Number of edges: 1
```

Example 7 — Search history

```
1. data   2. code   3. dsa   4. hello
Removed search: data
Latest search: code
Number of searches: 3
```

Example 8 — Recent searches

```
1. dsa   2. code
Removed oldest search: dsa
Removed latest search: code
Recent search count: 0
```

Example 9 — Sparse matrix

```
Document: dsa.txt | Word: a    | Frequency: 2
Document: dsa.txt | Word: and  | Frequency: 1
Document: dsa.txt | Word: data | Frequency: 3  ...
```

22. Testing

ID	Feature	Input	Expected output	Actual output	Status
T01	Index construction	Start program	“Search index built successfully.”	Same	Pass
T02	Word search	`data`	Documents containing `data`, ranked by score	dsa.txt (3), algorithms.txt (3)	Pass
T03	Unknown word	`hello`	“No documents found.”	Same	Pass
T04	Vocabulary	Menu 2	84 unique words, sorted	84 words, sorted	Pass

T05	Linear search	`data`	Found	Found, 18 comparisons	Pass
T06	Binary search	`data`	Found	Found, 5 comparisons	Pass
T07	Bubble sort	Menu 4→1	n(n−1)/2 = 3486	3486	Pass
T08	Selection sort	Menu 4→2	3486	3486	Pass
T09	Insertion sort	Menu 4→3	83 on sorted input	83	Pass
T10	Display graph	Menu 5→1	4 adjacency lists with weights	As in Section 14	Pass
T11	BFS	start 1	dsa, database, algorithms, cpp	Same	Pass
T12	DFS	start 1	dsa, database, algorithms, cpp	Same	Pass
T13	Shortest path	1 → 4	dsa.txt → database.txt, 1 edge	Same	Pass
T14	Search history	Menu 6	Newest first	data, code, dsa, hello	Pass
T15	Remove latest history	Menu 6→2	Removes newest	“Removed search: data”	Pass
T16	Recent searches	Menu 7	Oldest first	dsa, code	Pass
T17	Remove oldest / latest	Menu 7→2, 7→3	dsa then code; count 0	Same	Pass
T18	Sparse matrix	Menu 8	Non-zero entries listed	Entries shown	Pass

23. Error Handling

Situation	Handling in the code
Invalid main-menu input	“Invalid choice.” then pause
Invalid submenu input	“Invalid choice.” (loop continues)
Empty or symbol-only search	“No valid search words found.”; not saved to history
Word not found	“No documents found.” / “Not Found” / “Word not found.”
Invalid word in algorithm menu	“Invalid word.”
Non-numeric document choice (graph)	`try/catch` around `stoi` → “Invalid document choice.”
Out-of-range document number	Range check; also guarded inside `Graph` methods
Empty history / deque	Checked with `empty()`; a message is printed
Empty vocabulary	“Vocabulary is empty.”
Missing document file	`processDocument` returns an empty list (no message shown)

24. Advantages

- Integrates many DSA topics in one runnable application.
- Modular design: one class per structure and a coordinating `SearchEngine`.
- The inverted index avoids rescanning documents at query time.
- The sorted vocabulary comes from the BST inorder traversal.
- Heap-based ranking returns the best match first.
- Comparison counters make algorithm costs visible and comparable.
- Destructors release all dynamically allocated memory.
- Simple console interface; only the C++ standard library is needed.

25. Limitations

- The document list is fixed (4 files).
- Console only; the index is in memory and rebuilt on every launch; history is not saved.
- Ranking is a raw frequency sum — no TF-IDF, no document-length normalisation, no stop-word removal, no stemming.
- Multi-word queries behave like OR; no phrase or Boolean search.
- First-letter bucketing is uneven, so the index lookup is not truly O(1).
- The BST is unbalanced; worst case O(V).
- Graph shortest path ignores edge weights; the sample graph is complete, so paths are trivial.
- Search functions pass vectors by value (extra copies); `sortData` is $O(n^2)$ and runs each time binary search is chosen.
- A missing document file is skipped silently.
- `C++` becomes `c` and hyphenated words are merged.

26. Future Improvements

GUI or web interface; dynamic loading of documents from a folder; a real hash table for the index; a self-balancing tree (AVL / Red-Black) for the vocabulary; TF-IDF ranking; AND / OR / phrase queries; stop-word removal and stemming; spelling correction and autocomplete (trie); Dijkstra’s algorithm using the edge weights; a CSR-style sparse matrix; persistent storage of the index;

limiting the history length; early-exit bubble sort; passing vectors by `const&`.

27. Coverage of DSA Topics

Topic	Implemented?	Where
Arrays	Yes	Bucket array, BFS queue, visited arrays, vectors
Linked lists	Yes	Index, sparse matrix, stack, deque, adjacency lists, `LinkedList` class
Trees	Yes	BST vocabulary
Graphs	Yes	Document graph, BFS, DFS, shortest path
Heaps	Yes	Max-heap ranking
Stack	Yes	Search history
Deque	Yes	Recent searches
Searching	Yes	Linear, binary, index lookup, BST
Sorting	Yes	Bubble, selection, insertion
Complexity analysis	Yes	Comparison counters and Section 18
OOP	Yes (no inheritance)	Classes, composition, destructors
Hash tables	Concept only	Bucketed index approximates the idea (Section 17)
Balanced trees, TF-IDF, Dijkstra	Future work	—

28. Conclusion

SearchX demonstrates how several fundamental data structures cooperate in a working application. Documents are cleaned by `TextProcessor`, indexed in an array-of-linked-lists inverted index and summarised in a BST vocabulary of 84 unique words. Queries are scored by term frequency and ranked with a max heap. Linear, binary and index lookups, and bubble, selection and insertion sort, are implemented with comparison counters whose results (18, 5, 3486, 3486 and 83) agree with the theoretical analysis. A weighted document graph supports BFS, DFS and shortest path; a sparse matrix stores document–term frequencies; a stack and a deque manage search history and recent searches.

The project also shows its limitations honestly — a fixed document set, a first-letter bucket index rather than a true hash table, basic ranking and an unbalanced BST — which point to clear future work. Overall, it shows practical use of arrays, linked lists, trees, graphs, heaps, stacks, deques, searching, sorting, complexity analysis and object-oriented programming in C++.

29. References

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, MIT Press.
2. M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, Pearson.
3. E. Horowitz, S. Sahni, S. Anderson-Freed, *Fundamentals of Data Structures in C++*, Silicon Press.
4. B. Stroustrup, *The C++ Programming Language* / *A Tour of C++*, Addison-Wesley.
5. C. D. Manning, P. Raghavan, H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press.
6. R. Sedgewick, K. Wayne, *Algorithms*, Addison-Wesley.
7. cppreference.com — C++ standard library documentation.

30. Appendix

A. Key code snippets

Text cleaning

```
string TextProcessor::cleanWord(string word) {
    string cleaned = "";
    for (int i = 0; i < word.length(); i++) {
        char ch = tolower(word[i]);
        if (isalnum(ch)) cleaned += ch;
    }
    return cleaned;
}
```

Index bucket function

```
int InvertedIndex::getIndex(string word) {
    if (word == "") return 26;
    char c = tolower(word[0]);
    if (c >= 'a' && c <= 'z') return c - 'a';
    return 26;
}
```

Ranking in search()

```
for each document i:
    totalScore = sum of getFrequency(queryWords[j], fileNames[i])
    if (totalScore > 0) resultHeap.insert(SearchResult(fileNames[i], totalScore));
while (!resultHeap.empty()) { result = resultHeap.extractMax(); /* print */ }
```

B. Important formulas

- Heap: parent = (i−1)/2, left child = 2i+1, right child = 2i+2.
- Bubble / selection sort comparisons: n(n−1)/2 → 84 × 83 / 2 = 3486.
- Insertion sort best case: n−1 comparisons → 83.
- Binary search worst case: $\lfloor \log_2 n \rfloor + 1$ → 7 for n = 84.
- Bucket index: `tolower(word[0]) – ‘a’`.
- Document score: Σ frequency(query word, document).
- Edge weight: |distinct words of D_i ∩ words of D_j|.